

1 Pseudocode Implementation

1.1 Policy Forward Pass

Algorithm 0: Action generation from the observation history

- 1: Receive the current observation o_t
- 2: Preprocess the observation
$$e_t = f_{\text{pre}}(o_t)$$
- 3: Update the recurrent hidden state
$$h_t = f_{\text{LSTM}}(e_t, h_{t-1})$$
- 4: Form the rolling window of recent LSTM hidden states
$$H_t = [h_{t-W+1}, \dots, h_t]$$
- 5: Process the window with the transformer block
$$z_t = f_{\text{tr}}(H_t)$$
- 6: Project the temporal feature and observation into the latent state
$$q_t = f_{\text{proj}}(z_t, o_t)$$
- 7: Compute regime mixture weights
$$\alpha_t = \text{softmax}(f_{\text{regime}}(q_t, e_t^{\text{exo}}))$$
- 8: Compute the actor-side dynamics feature d_t from the switching latent dynamics model
- 9: Form the policy input
$$p_t = [q_t, \alpha_t, d_t]$$
- 10: Compute the bounded stochastic policy distribution
$$\mu_t, \sigma_t = f_{\text{actor}}(p_t)$$
- 11: Sample the supervisory setpoint action
$$a_t = \left[\begin{matrix} P_{\text{suc,ref},t} \\ T_{\text{cab,ref},t} \end{matrix} \right] \sim \pi_{\theta}(\cdot | p_t)$$
- 12: Apply a_t to the refrigeration environment and store the transition

1.2 Constrained PPO Batch Update

Algorithm 1: Lagrangian PPO update used in the active code

- 1: Receive one collected trajectory batch
- 2: Read reward sequence r_t and weighted-temperature sequence $T_{\text{weighted},t}$
- 3: Convert weighted temperature into constraint cost
$$c_t = \max(T_{\min} - T_{\text{weighted},t}, T_{\text{weighted},t} - T_{\max}, 0)$$
- 4: For a generic signal x_t with value critic V^x , compute
$$\delta_t^x = x_t + \gamma_x V^x(o_{t+1}) - V^x(o_t)$$
- 5: Compute generalized advantage estimation
$$\hat{A}_t^x = \sum_{l=0}^{T-t-1} (\gamma_x \lambda_{\text{GAE},x})^l \delta_{t+l}^x$$
- 6: Compute the return target
$$\hat{R}_t^x = \hat{A}_t^x + V^x(o_t)$$
- 7: Apply the same GAE procedure to r_t using $V_{\phi}^r(o_t)$
- 8: Apply the same GAE procedure to c_t using $V_{\psi}^c(o_t)$
- 9: Form the constrained advantage
$$\hat{A}_t^{\text{constr}} = \text{normalize}(\hat{A}_t^r - \lambda \hat{A}_t^c)$$
- 10: Update the Lagrange multiplier
$$\lambda \leftarrow \text{clip}\left(\lambda + \eta_{\lambda} \left(\frac{1}{T} \sum_{t=0}^{T-1} c_t - d\right), 0, \lambda_{\max}\right)$$
- 11: **for** each PPO epoch **do**
- 12: **for** each recurrent minibatch **do**
- 13: Update the actor using the standard clipped PPO objective, replacing the usual reward advantage with $\hat{A}_t^{\text{constr}}$
- 14: Update the reward critic toward $\hat{R}_t^r$
- 15: Update the cost critic toward $\hat{R}_t^c$
- 16: Update the latent rollout and temperature-prediction model using Algorithm 2
- 17: **end for**
- 18: **end for**

1.3 Latent Rollout Update

Algorithm 2: Temporal encoding, latent rollout, and violation prediction

- 1: Read observation sequence o_t , collected actions a_t , and measured weighted-temperature targets $T_{\text{weighted},t}$
- 2: Extract selected exogenous inputs e_t^{exo} from the observation sequence
- 3: Encode observations through the same temporal path used by the actor
$$e_t = f_{\text{pre}}(o_t), \quad h_t = f_{\text{LSTM}}(e_t, h_{t-1})$$

$$H_t = [h_{t-W+1}, \dots, h_t], \quad z_t = f_{\text{tr}}(H_t)$$

$$q_t = f_{\text{proj}}(z_t, o_t)$$
- 4: Encode the observed future trajectory with the same encoder to obtain target latent states
$$q_{t+h} = f_{\text{proj}}(z_{t+h}, o_{t+h}), \quad h = 1, \dots, H$$
- 5: Compute measured violation targets
$$c_{t+h} = \max(T_{\min} - T_{\text{weighted},t+h}, T_{\text{weighted},t+h} - T_{\max}, 0)$$
- 6: Predict the one-step latent transition
$$\hat{q}_{t+1} = f_{\text{dyn}}(q_t, a_t, e_t^{\text{exo}})$$
- 7: Decode the one-step temperature residual
$$\Delta \hat{T}_{t+1} = f_{\text{temp}}(\hat{q}_{t+1}, e_{t+1}^{\text{exo}})$$
- 8: Compute the one-step weighted-temperature prediction
$$\hat{T}_{t+1} = T_{\text{base},t} + \Delta \hat{T}_{t+1}$$
- 9: Compute predicted one-step violation
$$\hat{c}_{t+1} = \max(T_{\min} - \hat{T}_{t+1}, \hat{T}_{t+1} - T_{\max}, 0)$$
- 10: Initialize the open-loop latent rollout
$$\hat{q}_t = q_t$$
- 11: **for** $h = 0$ to $H - 1$ **do**
- 12: Propagate the latent state using the collected future action and exogenous input
$$\hat{q}_{t+h+1} = f_{\text{dyn}}(\hat{q}_{t+h}, a_{t+h}, e_{t+h}^{\text{exo}})$$
- 13: Decode the future temperature residual
$$\Delta \hat{T}_{t+h+1} = f_{\text{temp}}(\hat{q}_{t+h+1}, e_{t+h+1}^{\text{exo}})$$
- 14: Compute the future weighted-temperature prediction
$$\hat{T}_{t+h+1} = T_{\text{base},t} + \Delta \hat{T}_{t+h+1}$$
- 15: Convert predicted weighted temperature into predicted violation
$$\hat{c}_{t+h+1} = \max(T_{\min} - \hat{T}_{t+h+1}, \hat{T}_{t+h+1} - T_{\max}, 0)$$
- 16: **end for**
- 17: Compute one-step latent loss
$$L_{\text{dyn},1} = \|\hat{q}_{t+1} - q_{t+1}\|_2^2$$
- 18: Compute rollout latent loss
$$L_{\text{dyn},H} = \frac{1}{H} \sum_{h=1}^H \|\hat{q}_{t+h} - q_{t+h}\|_2^2$$
- 19: Compute one-step temperature loss
$$L_{T,1} = \|\hat{T}_{t+1} - T_{\text{weighted},t+1}\|_2^2$$
- 20: Compute rollout temperature loss
$$L_{T,H} = \frac{1}{H} \sum_{h=1}^H \|\hat{T}_{t+h} - T_{\text{weighted},t+h}\|_2^2$$

21: Compute one-step violation loss

$$L_{c,1} = \|\hat{c}_{t+1} - c_{t+1}\|_2^2$$

22: Compute rollout violation loss

$$L_{c,H} = \frac{1}{H} \sum_{h=1}^H \|\hat{c}_{t+h} - c_{t+h}\|_2^2$$

23: Add persistence-baseline penalty if the learned prediction is worse than persistence

24: Combine the active auxiliary losses

$$L_{\text{aux}} = \alpha_1 L_{\text{dyn},1} + \alpha_H L_{\text{dyn},H} + \beta_1 L_{T,1} + \beta_H L_{T,H} \\ + \gamma_1 L_{c,1} + \gamma_H L_{c,H} + L_{\text{persistence}}$$

25: Update the temporal encoder, latent projector, switching dynamics model, and temperature decoder using L_{aux}

1.4 Model Size in the Active Configuration

Part	Layer sizes / structure
Observation input	$o_t \in \mathbb{R}^{n_o}$, where n_o is read from environment reset
Action output	$a_t \in \mathbb{R}^2$ supervisory setpoints
Preprocessing network	$f_{\text{pre}} : n_o \rightarrow 256 \rightarrow 256$
LSTM memory	f_{LSTM} : input 256, hidden 256
Rolling window	$H_t = [h_{t-W+1}, \dots, h_t]$, with $W = 6$
Transformer block	f_{tr} : 2 layers, 4 heads, feed-forward dimension 512
Latent projector	$f_{\text{proj}} : (256 + n_o) \rightarrow 128 \rightarrow 128 \rightarrow 32$
Latent state	$q_t \in \mathbb{R}^{32}$
Switching regimes	$K = 4$ local latent regimes
Regime-weight network	$f_{\text{regime}} : (32 + n_{\text{exo}}) \rightarrow 128 \rightarrow 4$, followed by softmax to produce $\alpha_t \in \mathbb{R}^4$
Dynamics matrices	$A_k : 32 \times 32$, $B_k : 32 \times 2$, $E_k : 32 \times n_{\text{exo}}$
Temperature decoder	$f_{\text{temp}} : (32 + n_{\text{exo}}) \rightarrow 128 \rightarrow 128 \rightarrow 1$
Policy feature	$p_t = [q_t, \alpha_t, d_t]$, dimension $2 \cdot 32 + 4 = 68$
Actor head	$f_{\text{actor}} : 68 \rightarrow 256 \rightarrow 256 \rightarrow 2n_a$
Reward critic	$V_{\phi}^r(o_t)$, feed-forward MLP to scalar value
Cost critic	$V_{\psi}^c(o_t)$, feed-forward MLP to scalar value

1.5 Active Training Settings

Setting	Value
Parallel environments	60
Simulation step	120 s
Control interval	600 s
Episode horizon in code	288 control steps
Forecast horizon	8 h
Training frames	10,000,000
Frames per batch	$288 \cdot 60 \cdot 2 = 34,560$
PPO epochs	10
Actor learning rate	$5 \cdot 10^{-5}$
Reward critic learning rate	$5 \cdot 10^{-5}$
Cost critic learning rate	$5 \cdot 10^{-5}$
Dynamics learning rate	$5 \cdot 10^{-5}$
Reward $\gamma/\lambda_{\text{GAE}}$	0.99/0.98
Cost $\gamma/\lambda_{\text{GAE}}$	0.99/0.98
PPO clipping	0.2
Entropy coefficient	10^{-4}
Initial λ	1
Maximum λ	30
Cost budget	0.04
Rollout horizon	$H = 10$ control steps
Runtime note	CUDA GPU intended; FMU execution also contributes to wall-clock runtime